LAB 8:

UNSUPERVISED LEARNING AND GANs

University of Washington, Seattle

Fall 2025

OUTLINE

Part 1: Unsupervised Learning

- Supervised vs unsupervised
- Unsupervised learning in with NN

Part 2: Generative Model Taxidermy

- FVBN
- Variational Autoencoder
- GAN

Part 3: Generative Adversarial Networks

- GAN architecture

Part 4: GAN Optimization and Applications

- Competing cost function
- Minmax game optimization
- GAN variations

Unsupervised Learning

Supervised vs Unsupervised

Unsupervised Learning in NN

Supervised vs Unsupervised Learning

Supervised

Data:
{x} x: inputs **WITH labels**

Neural Network Goal:
Minimize specific **error**

Examples: Classification,
Regression, Detection, Prediction

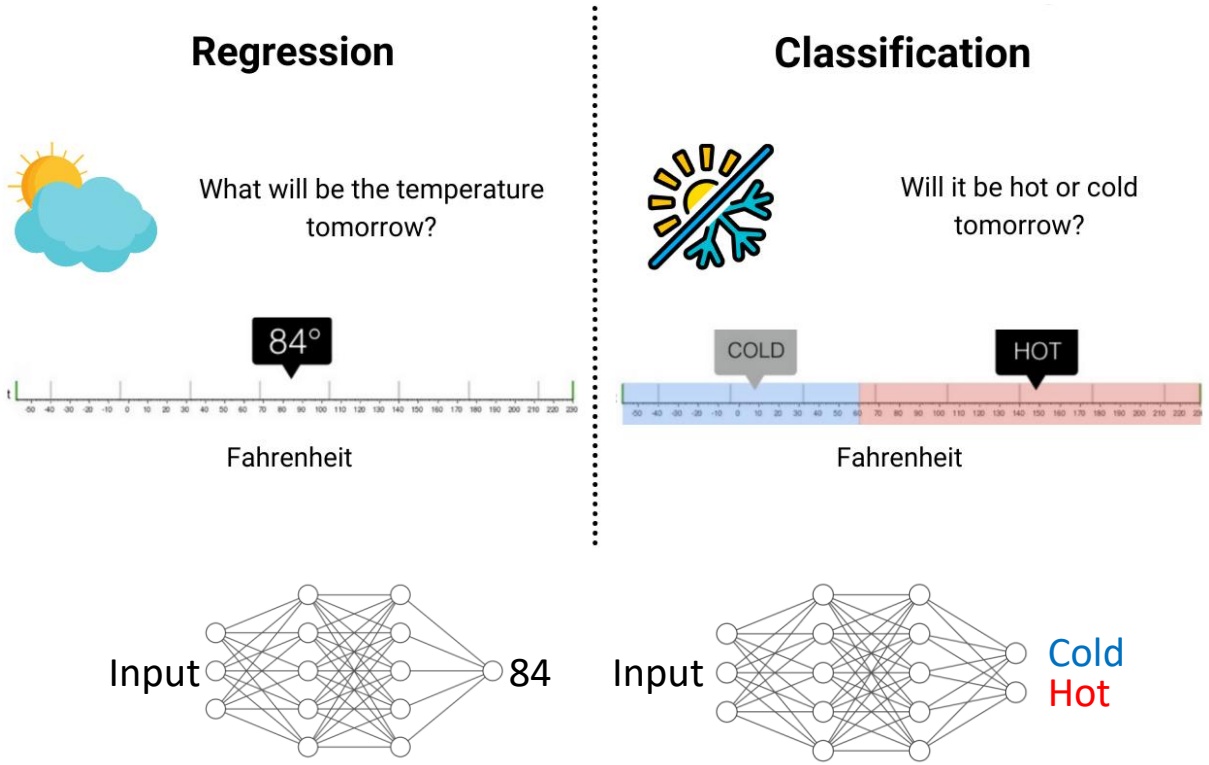
Supervised vs Unsupervised Learning

Supervised

Data:
{x} x: inputs **WITH labels**

Neural Network Goal:
Minimize specific **error**

Examples: Classification, Regression, Detection, Prediction



Unsupervised Learning in NN

Unsupervised

Data:

{x} x: inputs **WITHOUT labels**

Neural Network Goal:

Learn a **structure** of the data

Unsupervised Learning in NN

Training Data

Training Data ~ $\mathbf{P_{data}(x)}$

Unsupervised Learning in NN

Training Data

Generated Samples

http://www.whichfaceisreal.com/

Training Data ~ $\mathbf{P_{data}(x)}$

Generate Samples ~ $\mathbf{P_{model}(x)}$

Unsupervised Learning in NN

Training Data

Generated Samples

http://www.whichfaceisreal.com/

Training Data ~ $\mathbf{P_{data}(x)}$

Generate Samples ~ $\mathbf{P_{model}(x)}$

Goal: Model estimated density ≈ Real world density

Core problem in unsupervised learning

Unsupervised Learning in NN

Unsupervised

Data:

{x} x: inputs **WITHOUT labels**

+ No need for labeling → More data

- Challenge: Cost?

Neural Network Goal:

Learn a **structure** of the data

+ Has the potential to learn the real world

- Challenge: Optimization?

Maximum Likelihood Estimation

Data points
(e.g., human weights)

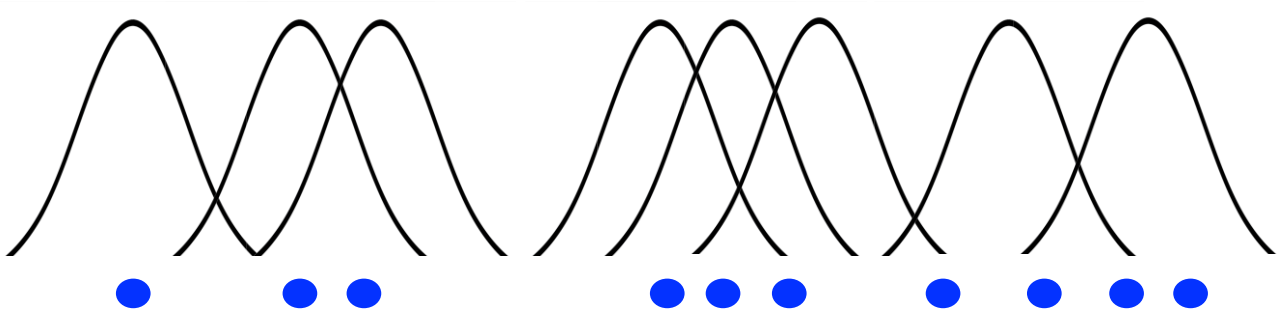
Maximum Likelihood Estimation

Guessed location of the distribution center

Data points
(e.g., human weights)

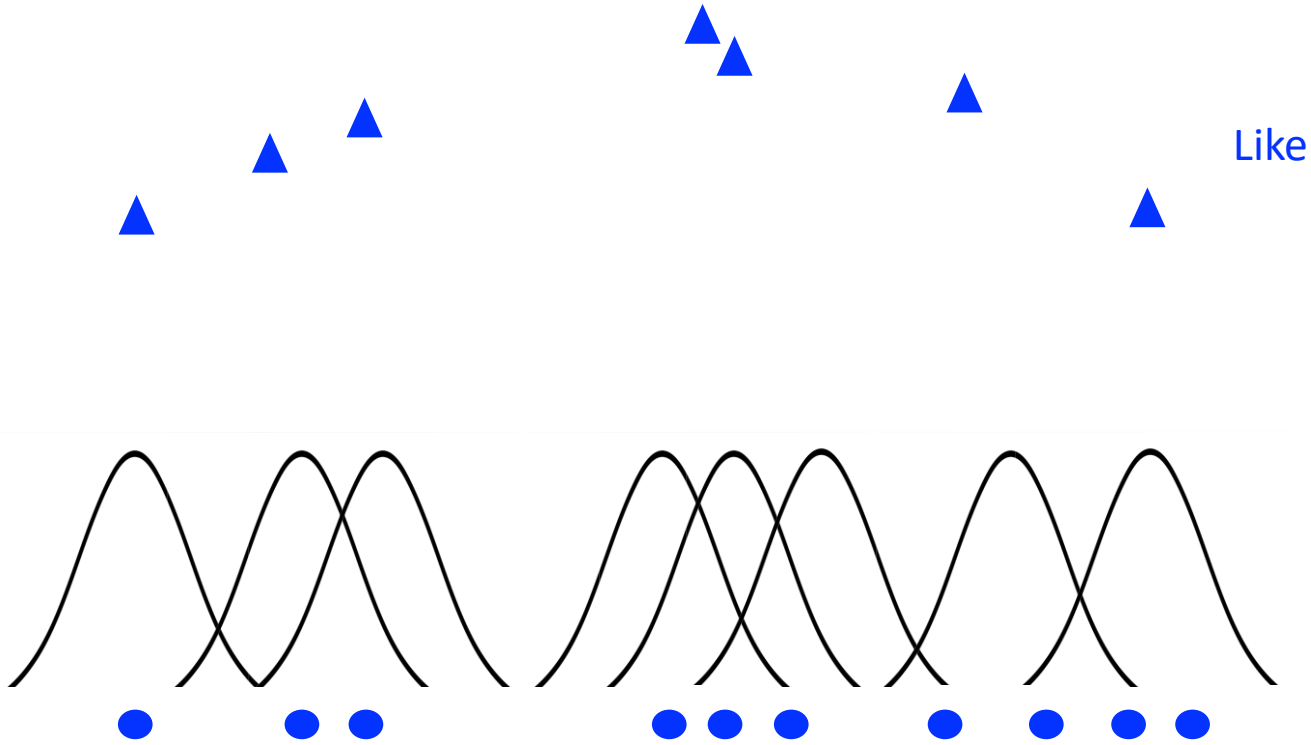
Maximum Likelihood Estimation

Location of the distribution center

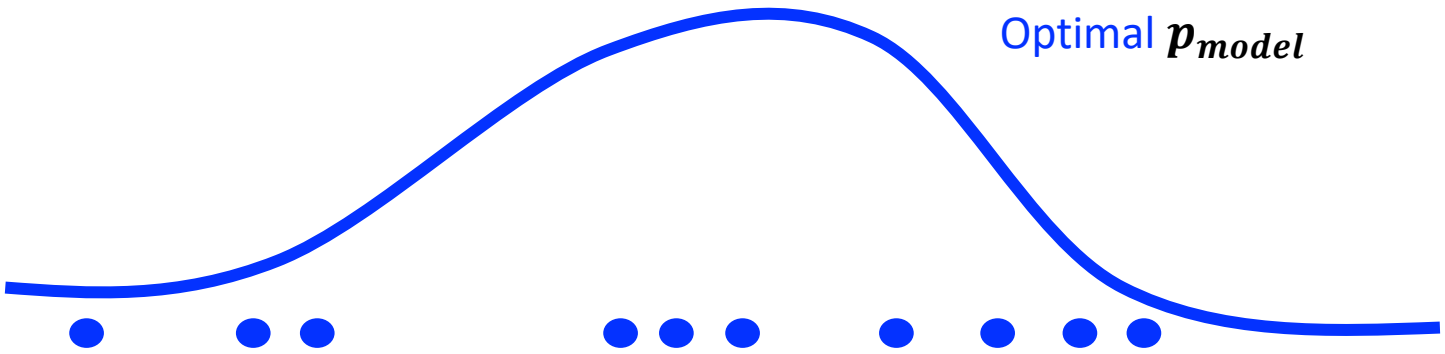


Data points
(e.g., human weights)

Maximum Likelihood Estimation



Unsupervised Learning in NN



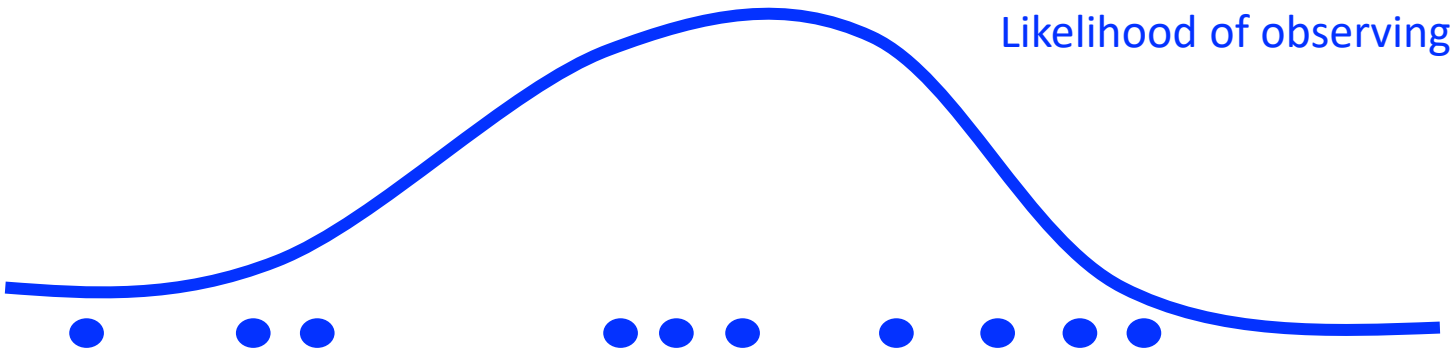
Model params

$$\theta^* = argmax_\theta \, E_{x \sim p_{data}} \, log \, p_{model}(x|\theta)$$

Real data

Guessed distribution

Unsupervised Learning in NN

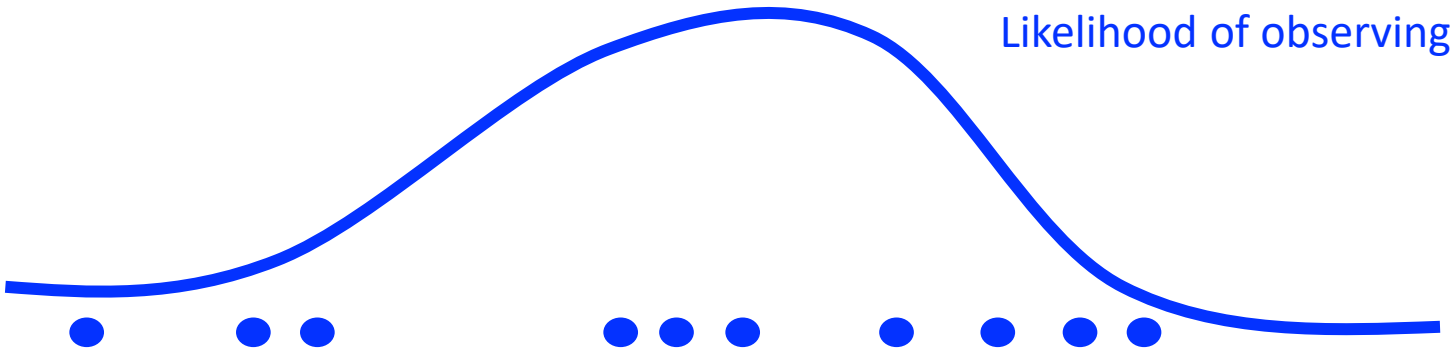


Model params

$$\boldsymbol{\theta^* = argmax_\theta \, E_{x \sim p_{data}} \, log \, p_{model}(x|\theta)}$$

Goal: Find the optimal distribution $\boldsymbol{p_{model}(x|\theta)}$ that best fit the data

Unsupervised Learning in NN



Model params

$$\theta^* = argmax_\theta \, E_{x \sim p_{data}} \, log \, p_{model}(x|\theta)$$

Explicit – explicitly define and generate P_{model}

Implicit - generate P_{model} without defining P_{model} exactly

Generative Model Taxidermy

Fully Visible Belief Network

Variational Autoencoder

Generative Adversarial Network

Unsupervised Learning in NN

Ian Goodfellow

Fully Visible Belief Network

- Explicitly formula based on chain rule:

$$\boldsymbol{p_{model}(x) = p_{model}(x_1)\prod_{i=2}^{n} p_{model}\,(x_i \,|x_1, x_2, \ldots, x_{i-1})}$$

- O(n) generation cost

- No control through hidden variables

Language Model

Language model: probability distribution over sequences of words. Given such a sequence, say of length m, it assigns a probability to the whole sequence.

Language Model

Language model: probability distribution over sequences of words. Given such a sequence, say of length m, it assigns a probability to the whole sequence.

P(W=NASA will **take** me to Moon) = p1
P(W= NASA will **bake** me to Moon) = p2
p2 << p1

Language Model

Language model: probability distribution over sequences of words. Given such a sequence, say of length m, it assigns a probability to the whole sequence.

P(W=NASA will **take** me to Moon) = p1
P(W= NASA will **bake** me to Moon) = p2
p2 << p1

Chain rule is used to estimate probability:

$$P(w_1 w_2 \ldots w_n) = \prod_i P(w_i | w_1 w_2 \ldots w_{i-1})$$

P(W) = P(**NASA**) P(**will**| NASA) P(**take**| NASA will) P(**me**| NASA will take)
P(**to**| NASA will take me) P(**Moon** | NASA will take me to)

Variational Autoencoder

$$\boldsymbol{p_{model}(x) = \prod_{i=2}^{n} p_{model}\left(x_i \,|x_1, x_2, \ldots, x_{i-1}\right)}$$

$$\boldsymbol{p_{model}(x) = \int p_{model}(z)\, p_{model}(x|z)\, dz}$$

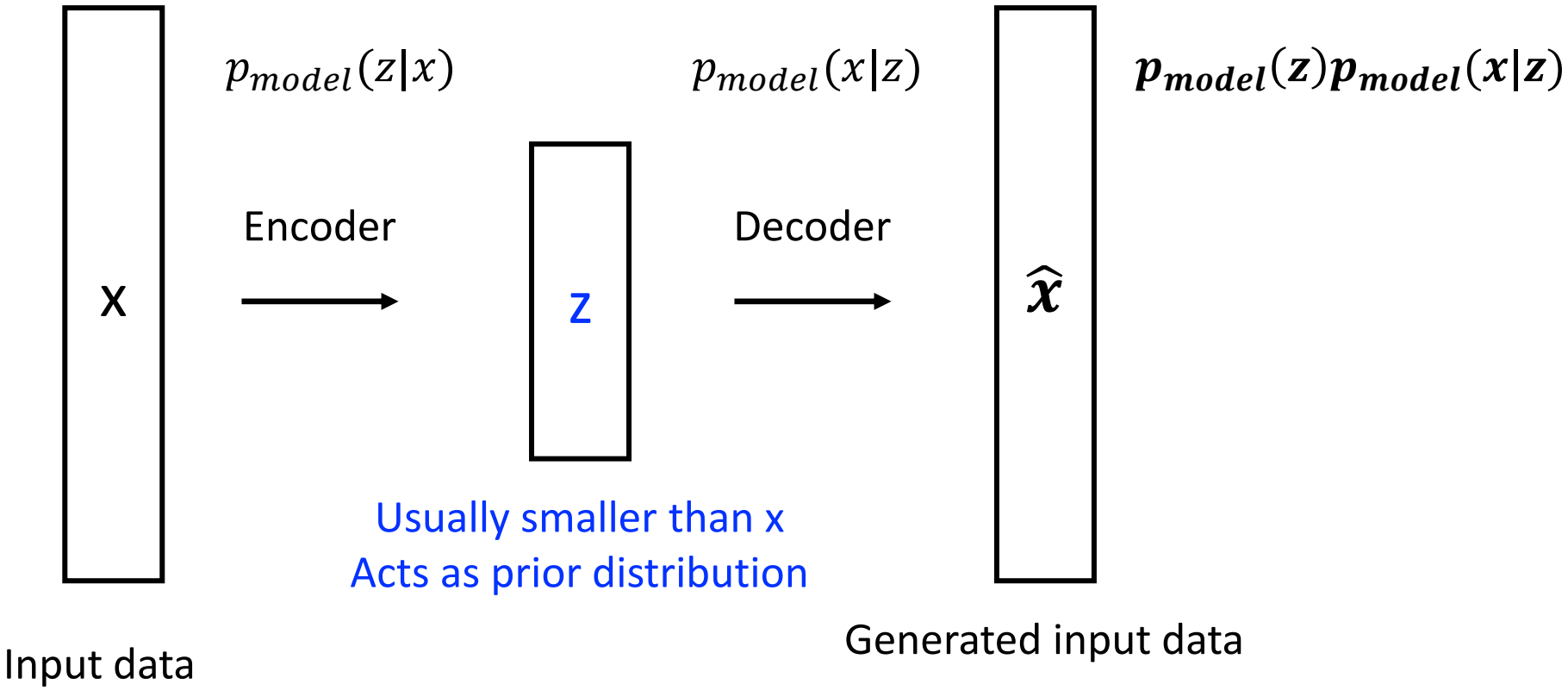
Variational Autoencoder

$$\boldsymbol{p_{model}(x) = \prod_{i=2}^{n} p_{model}\left(x_i \,|x_1, x_2, \ldots, x_{i-1}\right)}$$

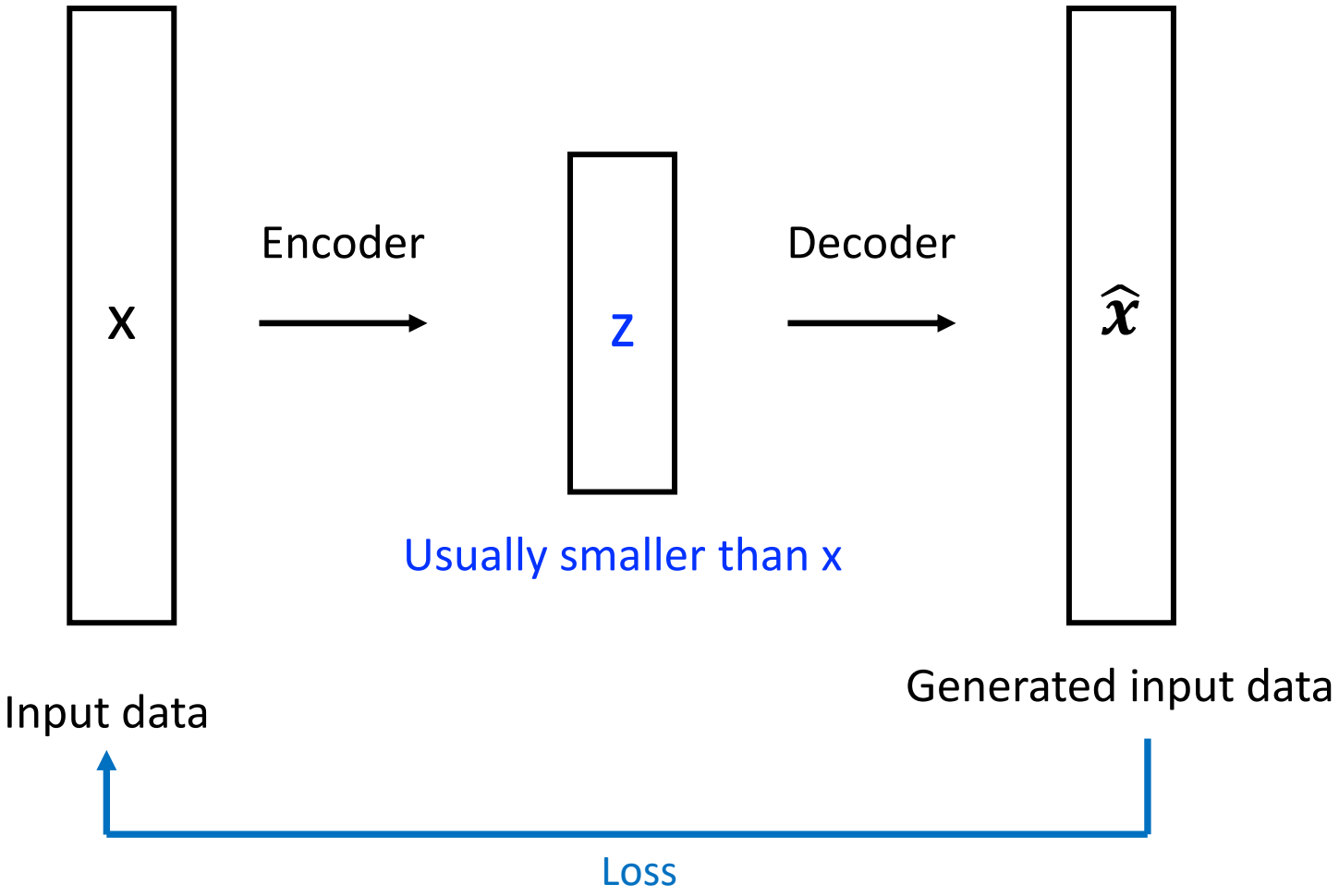
$$\boldsymbol{p_{model}(x) = \int p_{model}(z)\, p_{model}(x|z)\, dz}$$

$\boldsymbol{p_{model}(x)}$ is controlled by hidden state $\boldsymbol{z}$

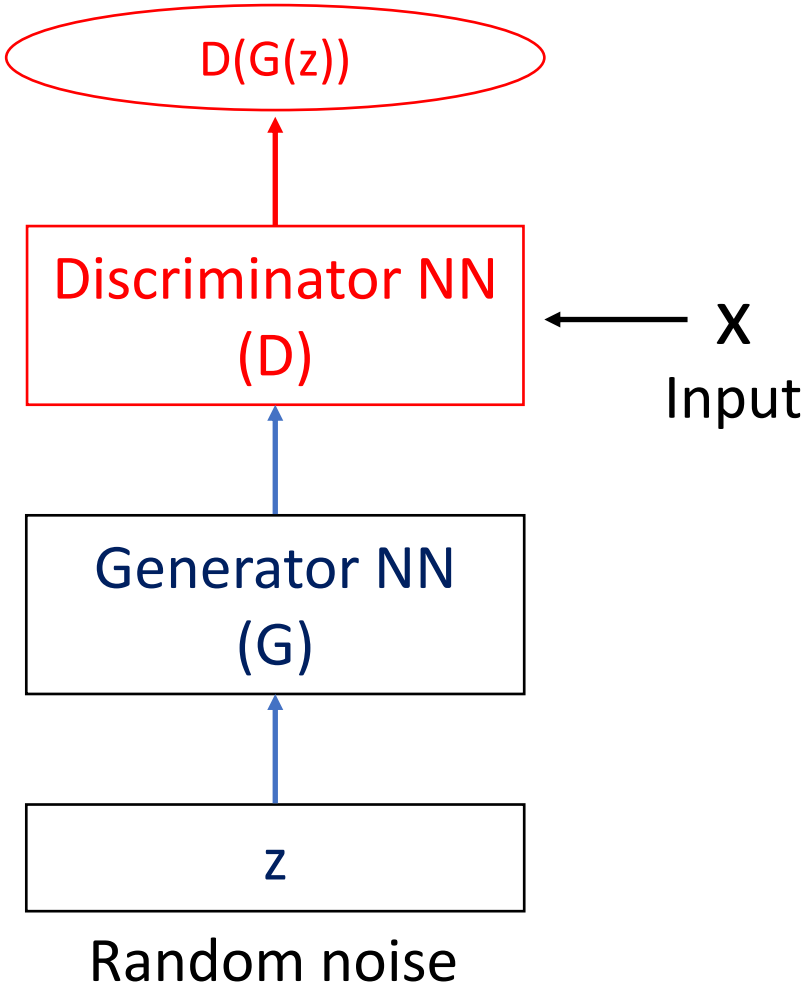
Variational Autoencoder



Variational Autoencoder



GAN



- Instead of sampling from **high dimensional, complex and unknown distribution**
- Sample from **simple distribution**, e.g. normal distribution (random noise) and **find transformation** to the distribution we want to learn.
- **Learn the transformation using a NN**

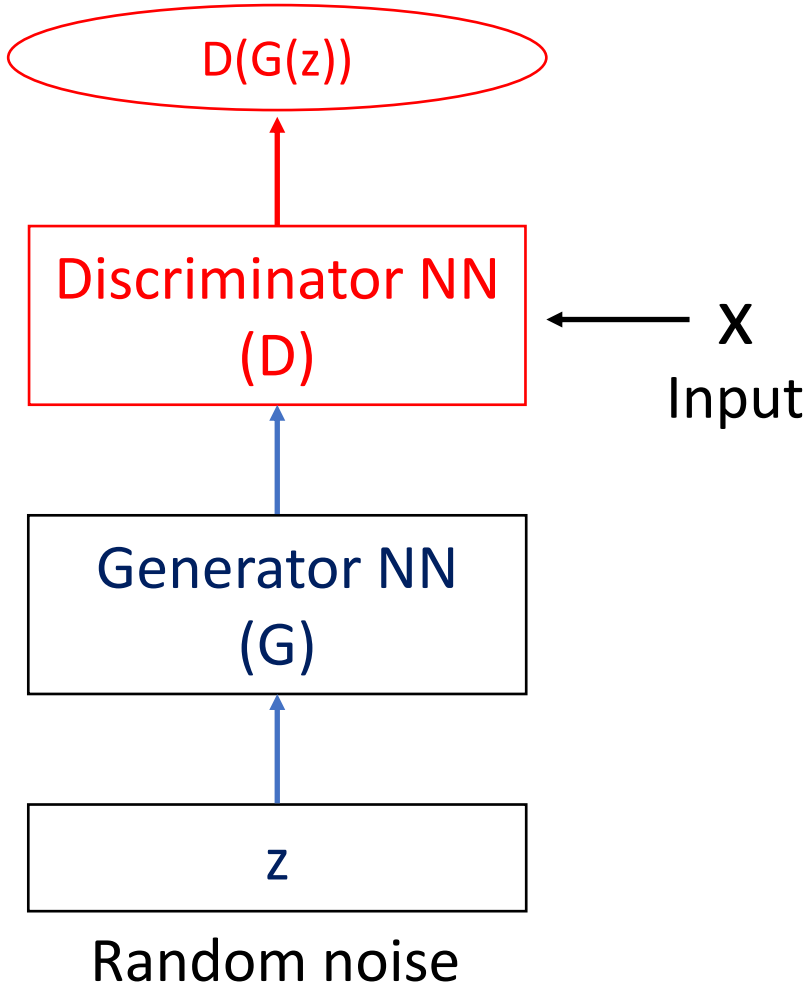
Generative Adversarial Networks

GAN Architecture

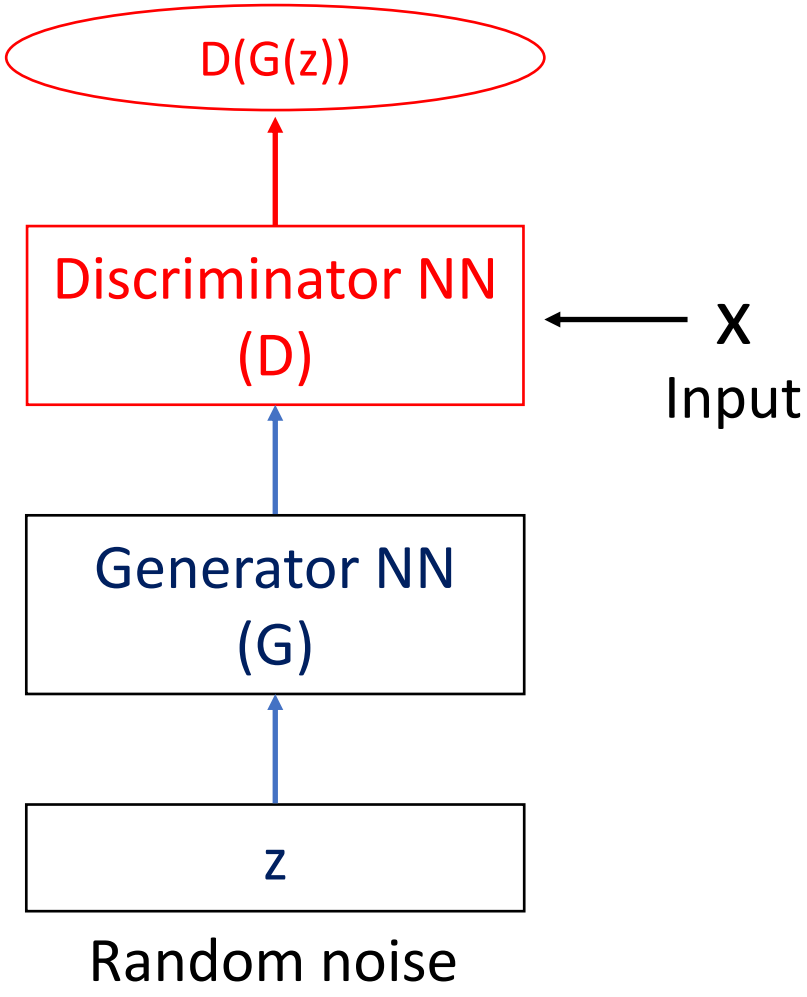
Generator Network

Discriminator Network

GAN



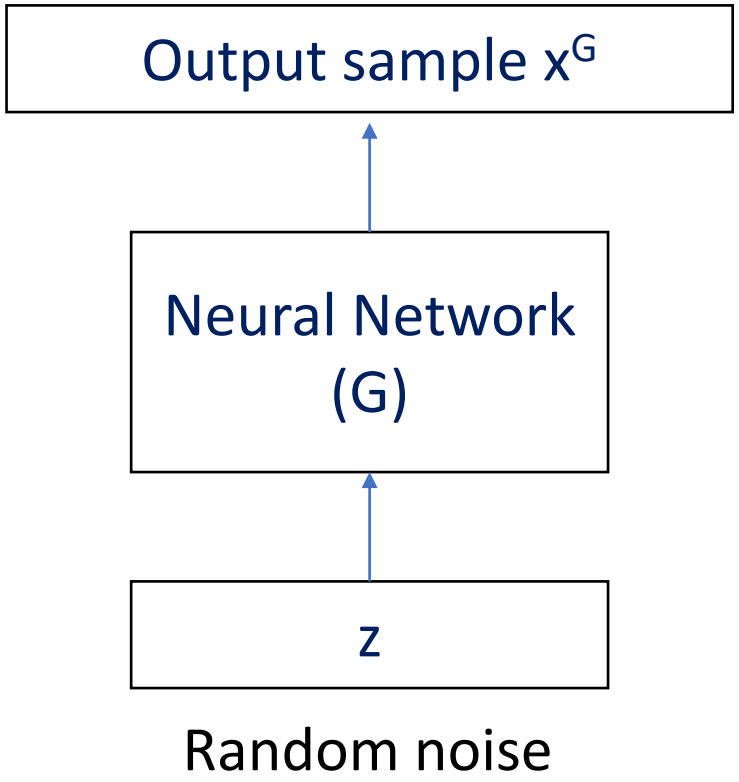
GAN



Discriminator – try to distinguish between **x (real) and generated (fake)** images

Generator – try to generate **samples and present them as real world** and fool the discriminator

Generator (G)



Training data has distribution $\mathbf{p_{data}}$.
Sample $\mathbf{x} \sim \mathbf{p_{data}}$.

Goal: Output sample $\mathbf{x^G}$ is of similar dimensions as **x** and distribution $\mathbf{p_{data}}$.

Examples

Face:

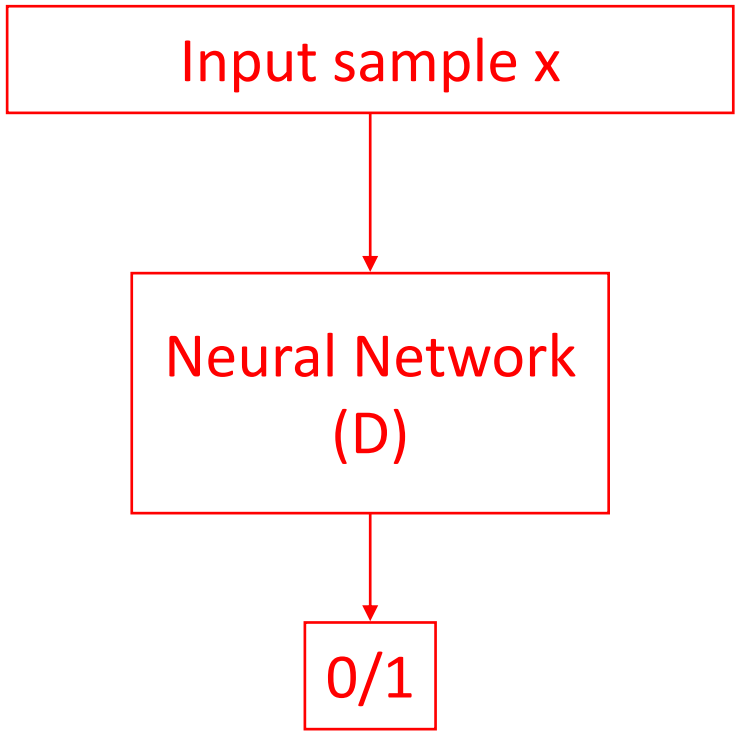
Car:

Bedroom:

Discriminator (D)

Receives input of same dimensions as $\mathbf{p_{data}}$.

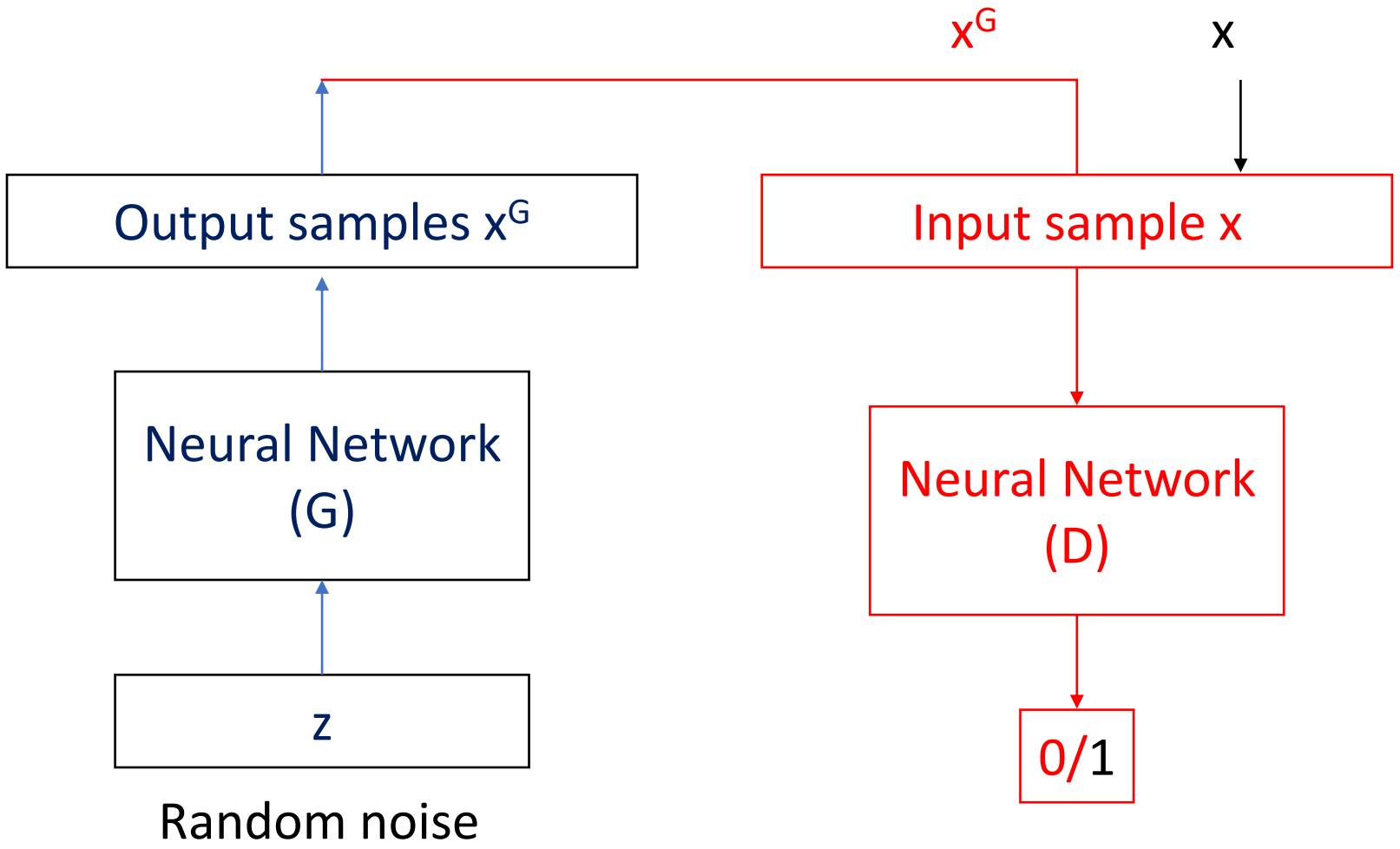
Goal: Distinguish sample from $\mathbf{p_{data}}$ (1) or not (0).



Examples

		Discriminator
Face (gen):		0
Car (real):		1
Bedroom (gen):		0

Full Architecture



GAN Optimization and Applications

Competing Loss Functions

Minmax Game Optimization

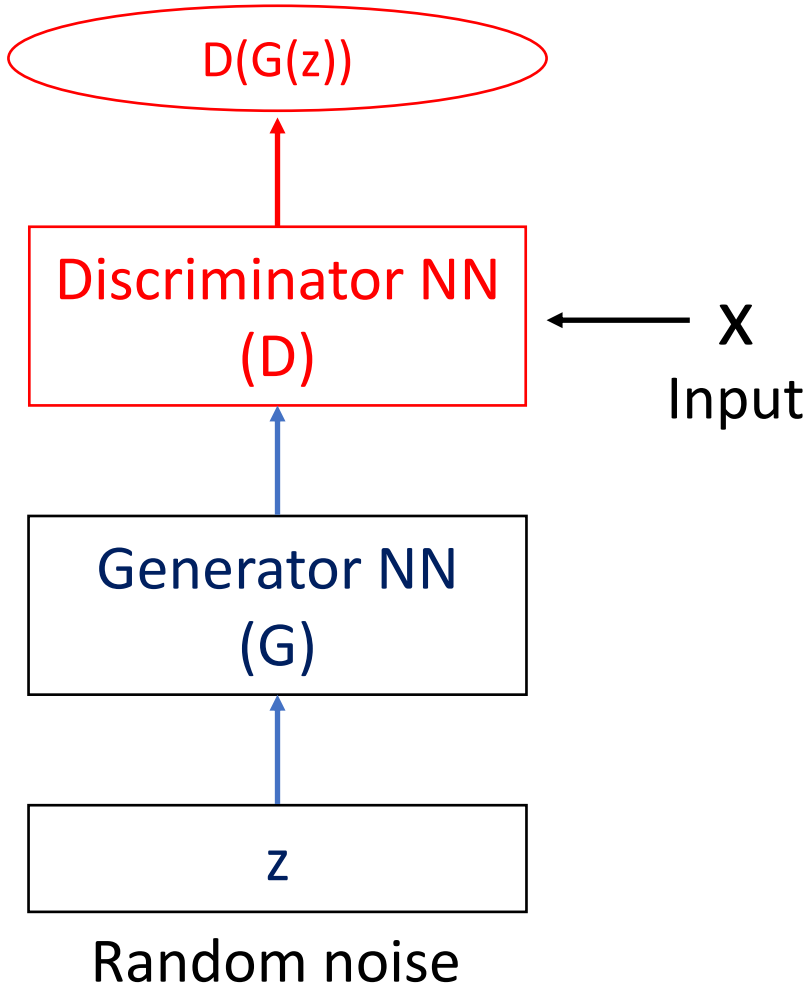
Optimization in NN

GAN Applications

Competing cost functions

"Cost" $\propto \|D(G(z))\|$

"Cost": $\propto \|1 - D(G(z))\|$



Competing cost functions

Binary Cross Entropy Loss

$$J^{(D)} = -\frac{1}{2}\mathbb{E}_{x\sim p_{data}} \log D_{\theta_d}(x) - \frac{1}{2}\mathbb{E}_{z\sim p_{model}} \log\left(1 - D_{\theta_d}(G_{\theta_g}(z))\right)$$

$$J^{(G)} = -J^{(D)}$$

$$J^{(D)} = -\frac{1}{2}\int \underbrace{p_{data}(x) \log D(x)}dx - \frac{1}{2}\int \underbrace{p_{model}(x) \log\big(1 - D(x)\big)}dx$$

Expected value of log probability that D correctly identifies x_{data} as real (1)

Expected value of log probability that D correctly identifies x_G as fake (0)

Minmax Game Optimization

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p_{model}} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

Discriminator output for real data

Discriminator output for generated data

Solution:

Saddle point in the parameter space (Nash Equillibrium)

- One player (Discriminator) is at **maximum**,
- Other player (Generator) is at **minimum**

Optimization in NN

- **Gradient ascent** for the discriminator on J

$$J^{(D)} = \frac{1}{2}\mathbb{E}_{x\sim p_{data}} \log D_{\theta_d}(x) + \frac{1}{2}\mathbb{E}_{z\sim p_{model}} \log\left(1 - D_{\theta_d}(G_{\theta_g}(z))\right)$$

$$\theta_d \leftarrow \arg\min_{\theta_d} J^{(D)}$$

- **Gradient descent** for the generator

$$J^{(G)} = -\frac{1}{2}\mathbb{E}_{z\sim p_{model}} \log\left(1 - D_{\theta_d}(G_{\theta_g}(z))\right)$$

$$\theta_g \leftarrow \arg\min_{\theta_g} J^{(G)}$$

Indirect learning via Discriminator

Optimal $D(x)$ is

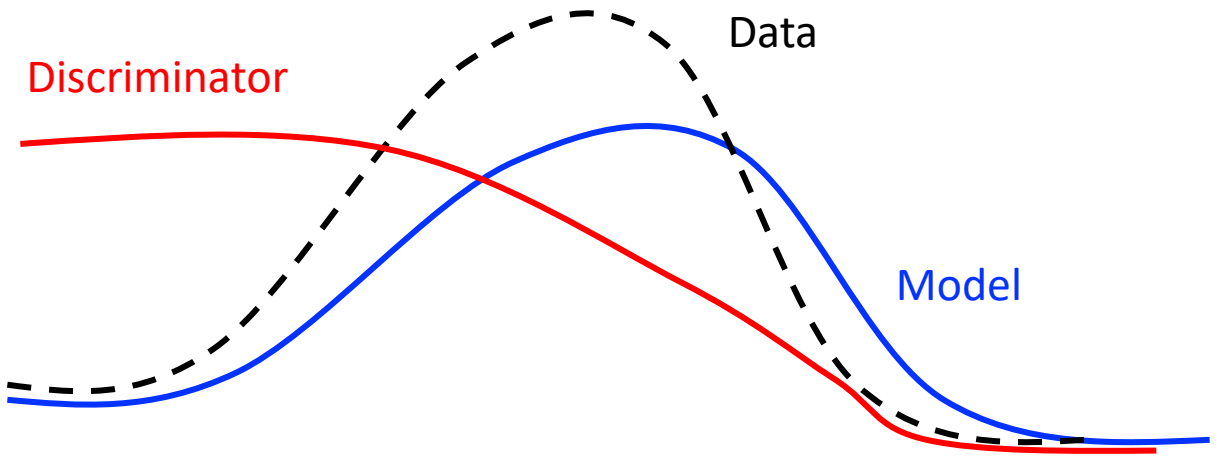
$$D(x) = \frac{p_{data}}{p_{model} + p_{data}}$$

Jensen-Shannon Divergence
(Generalization of KL-divergence)

Assumption: p_{model}, p_{data} are nonzero everywhere

Equilibrium: $p_{model} = p_{data}$ then $\mathrm{E}(D(x)) = \frac{1}{2}$

Indirect learning via Discriminator



Discriminator learns an approximation of $p_{data}(x)/p_{model}(x)$

vs

learning p_{model} (x) directly (or indirectly via latent variable models).

Optimization in NN

Take ***k*** gradient steps for the discriminator (k a hyperparameter), each doing the following:

- Sample *m* noise samples, $\{z^{(1)},z^{(2)},...,z^{(m)}\}$ from $p_{model}(z)$.
- Sample *m* actual samples, $\{x^{(1)},x^{(2)},...,x^{(m)}\}$ from $p_{data}(x)$: (a minibatch of your input data.)
- Perform an optimization step on the discriminator:

Optimization in NN

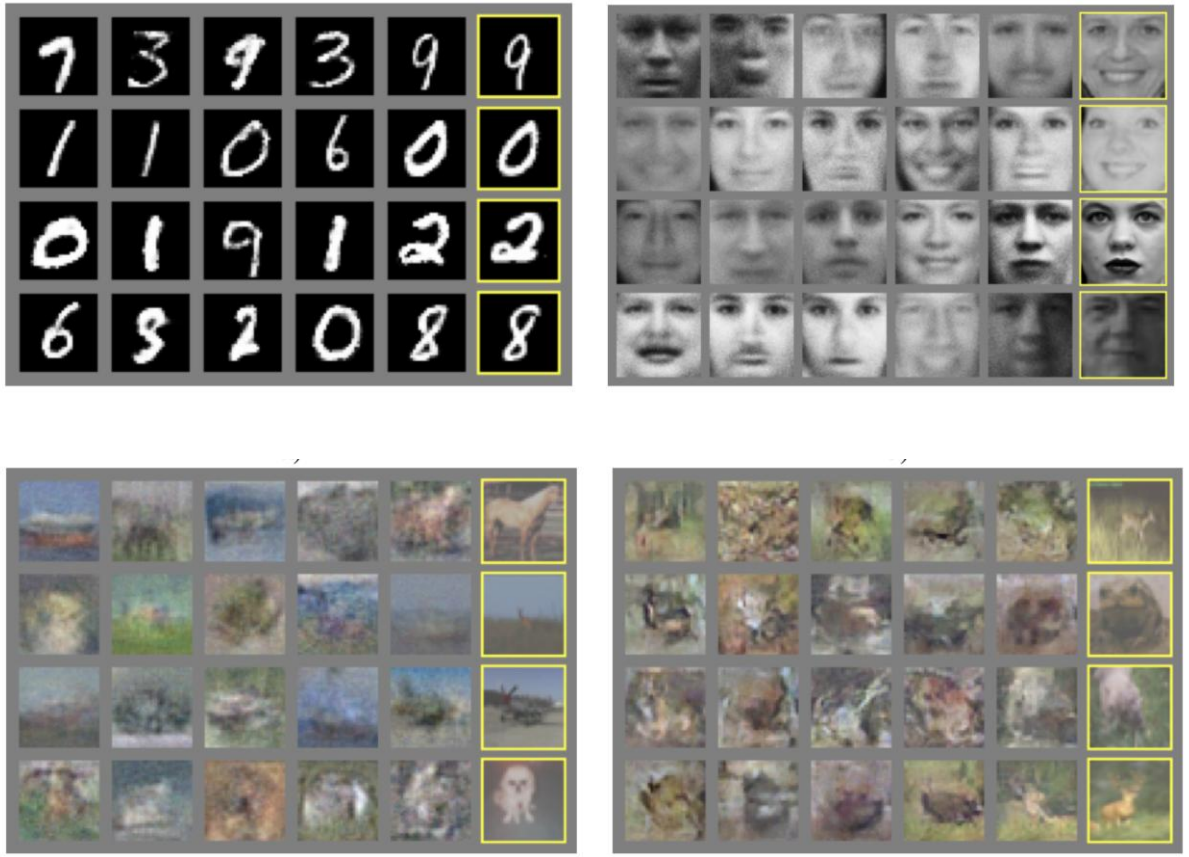
Take ***k*** gradient steps for the discriminator (k a hyperparameter), each doing the following:

- Sample *m* noise samples, $\{z^{(1)},z^{(2)},...,z^{(m)}\}$ from $p_{model}(z)$.
- Sample *m* actual samples, $\{x^{(1)},x^{(2)},...,x^{(m)}\}$ from $p_{data}(x)$: (a minibatch of your input data.)
- Perform an optimization step on the discriminator:

Do gradient descent step for the generator:

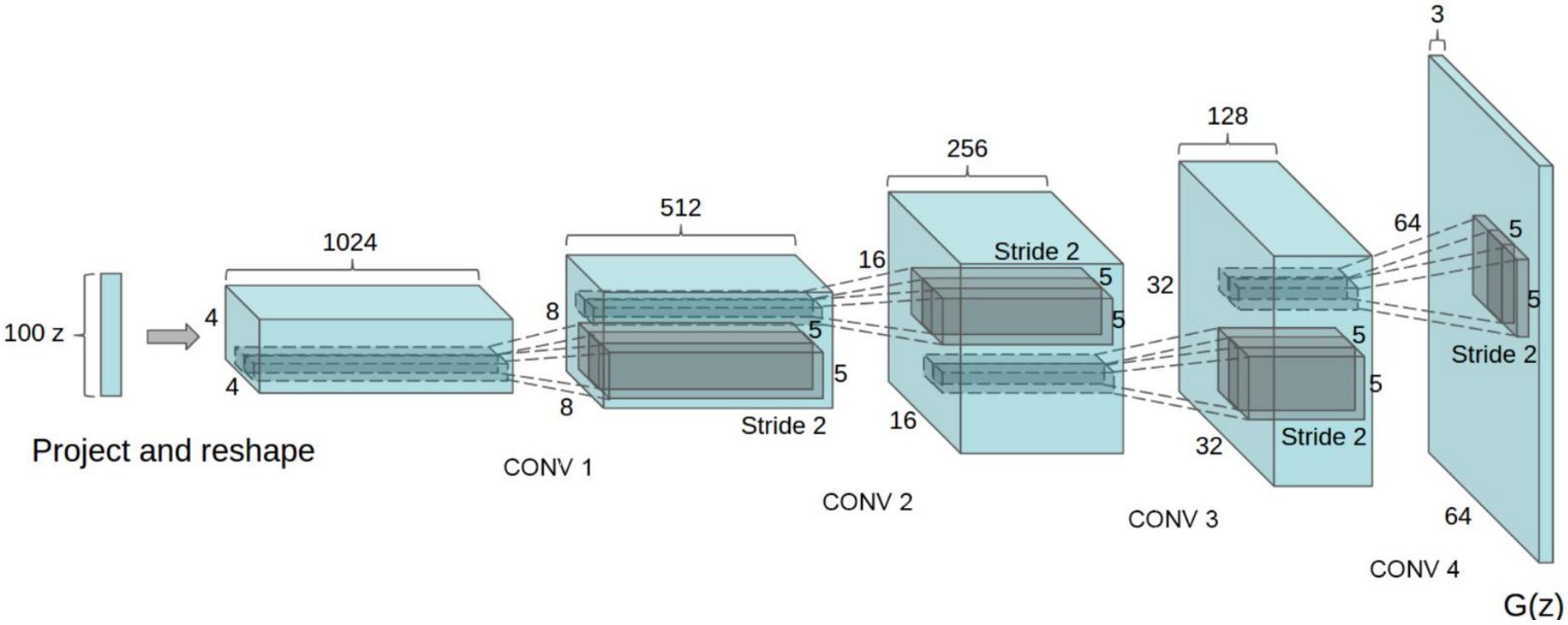
- Sample *m* noise samples, $\{z^{(1)},z^{(2)},...,z^{(m)}\}$ from $p_{model}(z)$.
- Perform an optimization step on the **generator**:

GAN Applications: Original GAN



Goodfellow et al. (2014) Generative Adversarial Nets

DCGAN



Radford, Alec, Luke Metz, and Soumith Chintala. "Unsupervised representation learning with deep convolutional generative adversarial networks." (2015).

Similarities in Hidden Space

woman with glasses

Generator hidden layers learn generic representation

Text to Image Synthesis

this small bird has a pink breast and crown, and black primaries and secondaries.

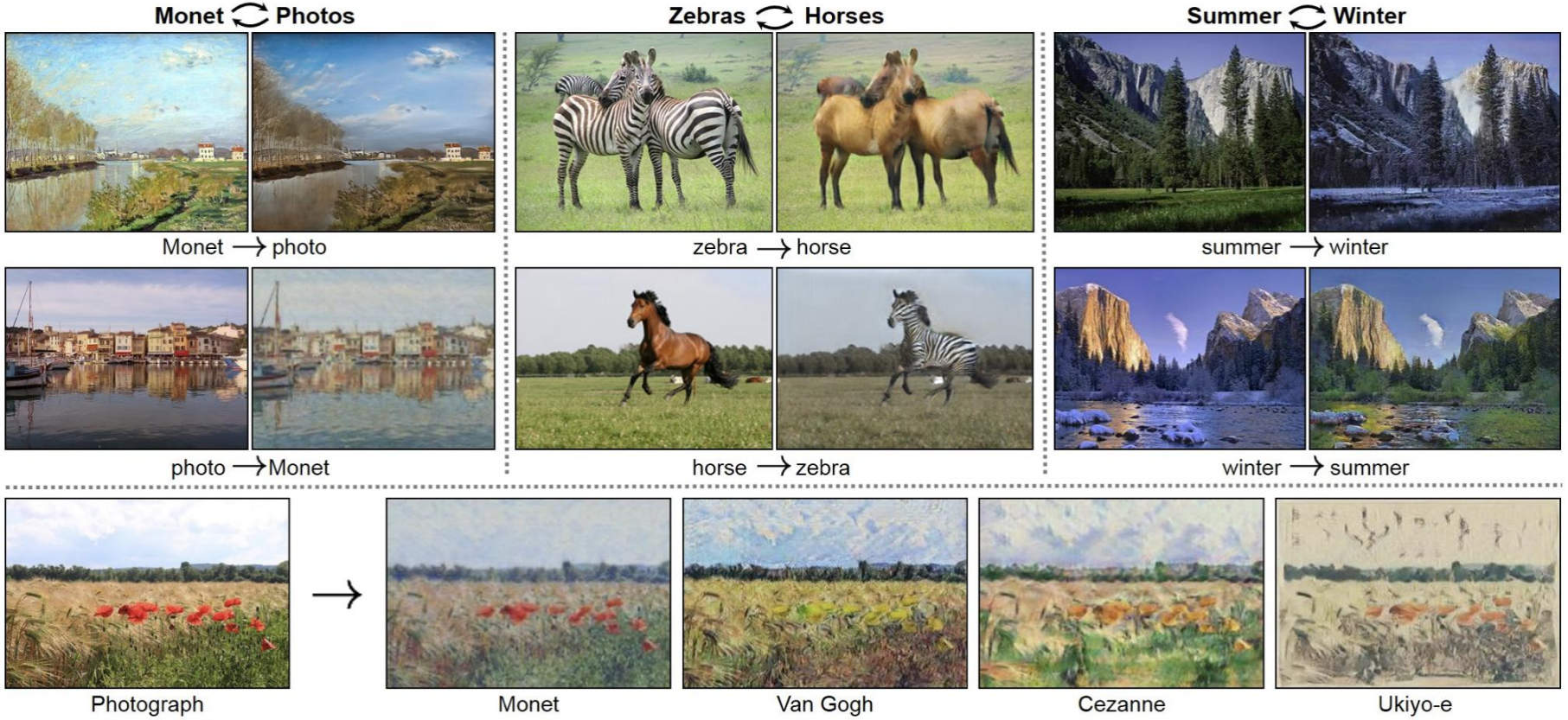
this magnificent fellow is almost all black with a red crest, and white cheek patch.

the flower has petals that are bright pinkish purple with white stigma

this white and yellow flower have thin white petals and a round yellow stamen

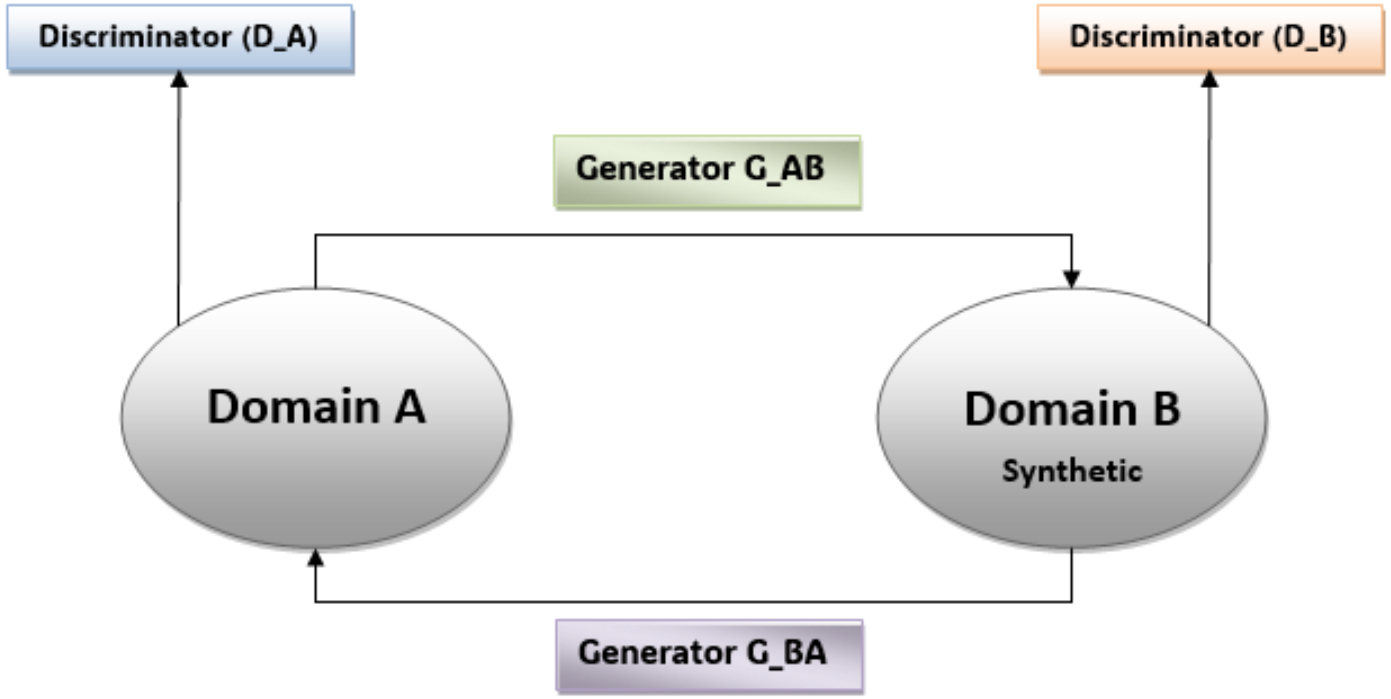
Reed et al. Generative Adversarial Text to Image Synthesis (2017)

CycleGAN

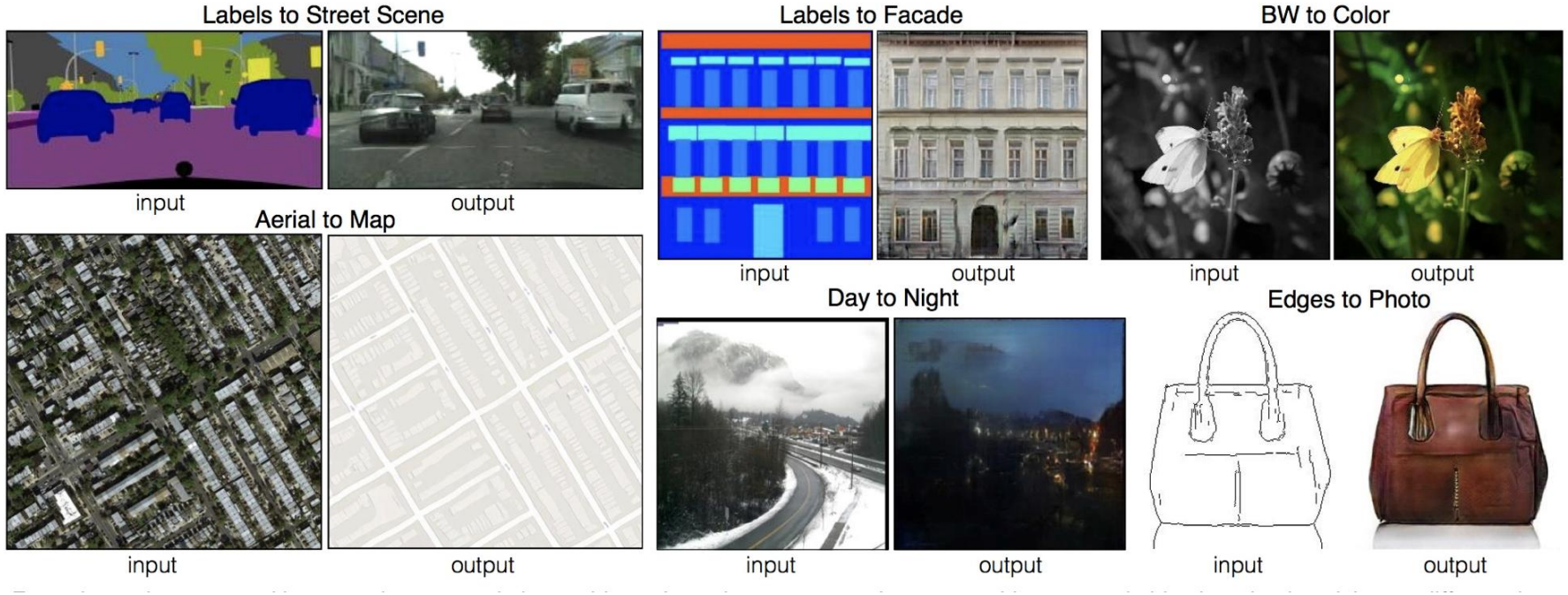


Zhu et al., Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks, ICCV 2017

CycleGAN


Discriminator (D_A)
Discriminator (D_B)
Generator G_AB
Domain A
Domain B
Synthetic
Generator G_BA

Pix2Pix

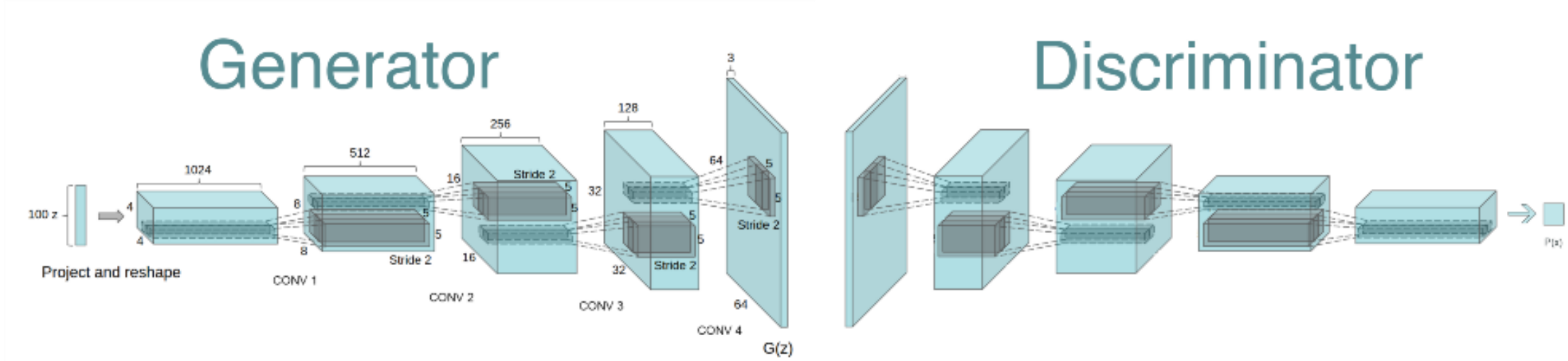


P. Isola et al. Image-to-Image Translation with Conditional Adversarial Nets, CVPR 2017

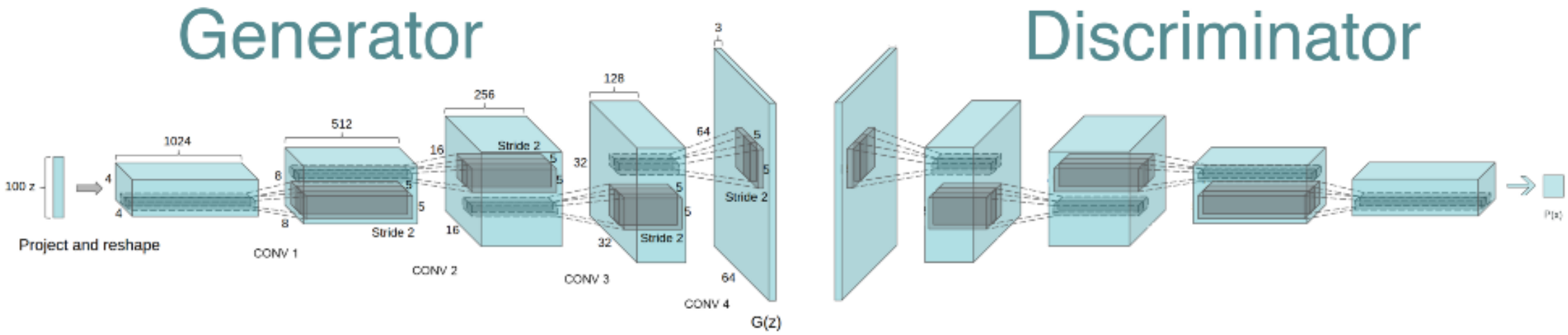
Generator Extension with Convolution:

2D Transpose Convolution

DCGAN Architecture



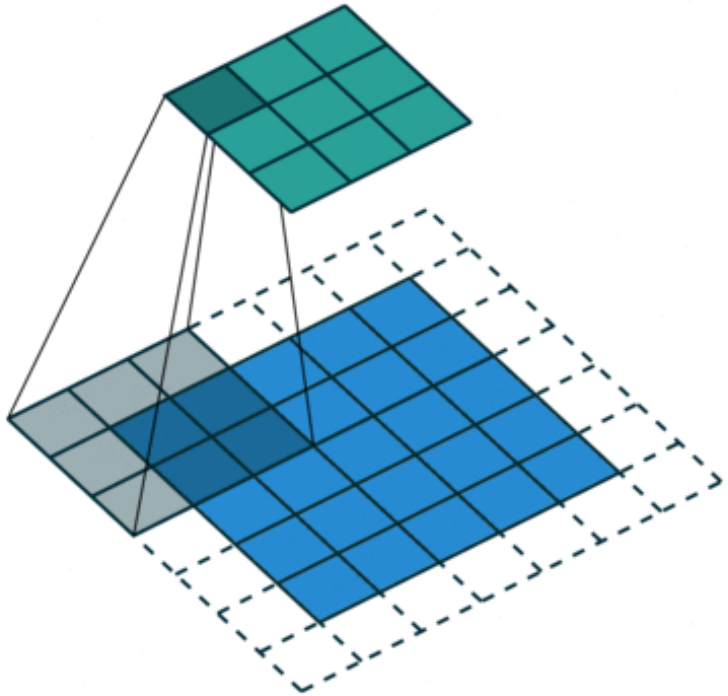
DCGAN Architecture



Layer (type)	Output Shape	Param #
ConvTranspose2d-1	[-1, 512, 4, 4]	819,200
BatchNorm2d-2	[-1, 512, 4, 4]	1,024
ReLU-3	[-1, 512, 4, 4]	0
ConvTranspose2d-4	[-1, 256, 8, 8]	2,097,152
BatchNorm2d-5	[-1, 256, 8, 8]	512
ReLU-6	[-1, 256, 8, 8]	0
ConvTranspose2d-7	[-1, 128, 16, 16]	524,288
BatchNorm2d-8	[-1, 128, 16, 16]	256
ReLU-9	[-1, 128, 16, 16]	0
ConvTranspose2d-10	[-1, 64, 32, 32]	131,072
BatchNorm2d-11	[-1, 64, 32, 32]	128
ReLU-12	[-1, 64, 32, 32]	0
ConvTranspose2d-13	[-1, 3, 64, 64]	3,072
Tanh-14	[-1, 3, 64, 64]	0

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 64, 32, 32]	3,072
LeakyReLU-2	[-1, 64, 32, 32]	0
Conv2d-3	[-1, 128, 16, 16]	131,072
BatchNorm2d-4	[-1, 128, 16, 16]	256
LeakyReLU-5	[-1, 128, 16, 16]	0
Conv2d-6	[-1, 256, 8, 8]	524,288
BatchNorm2d-7	[-1, 256, 8, 8]	512
LeakyReLU-8	[-1, 256, 8, 8]	0
Conv2d-9	[-1, 512, 4, 4]	2,097,152
BatchNorm2d-10	[-1, 512, 4, 4]	1,024
LeakyReLU-11	[-1, 512, 4, 4]	0
Conv2d-12	[-1, 1, 1, 1]	8,192
Sigmoid-13	[-1, 1, 1, 1]	0
Flatten-14	[-1, 1]	0

Conv2D vs ConvTranspose2D

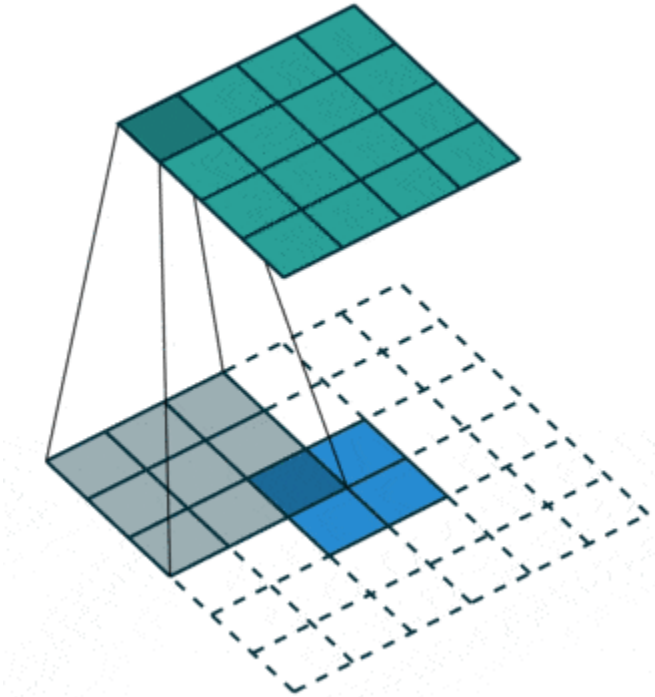


Conv2D()

input image = (5, 5)

Kernel size = (3, 3)

Output image = (3, 3)



ConvTranspose2D()

input image = (2, 2)

Kernel size = (3, 3)

Output image = (4, 4)

Conv2D vs ConvTranspose2D

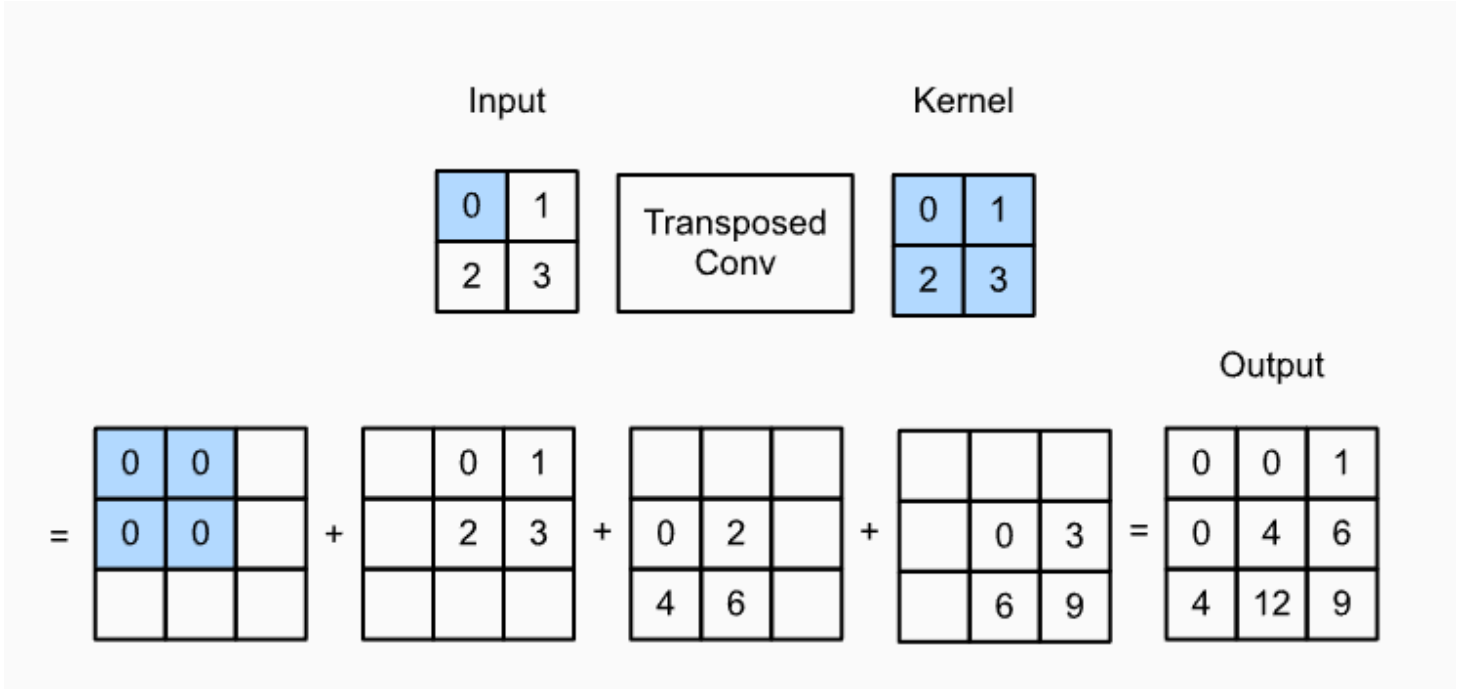
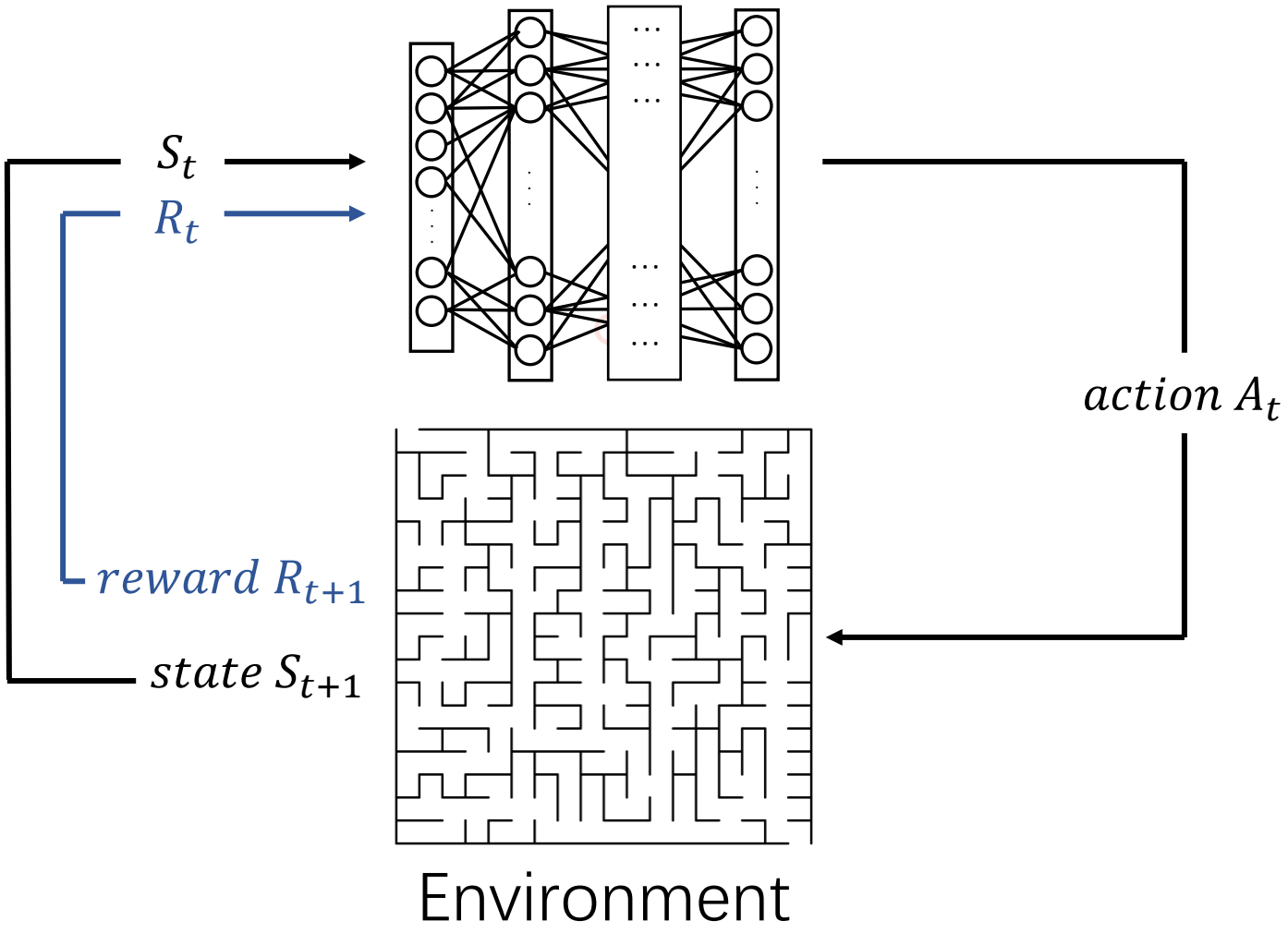


Image credit: d2l.ai

Next episode in EE P 596…



Deep Reinforcement Learning